

OOP INTO 1064 Lab - 2

From Biological to Mathematical Equations: Applying C++ Programming

Introduction:

This lab session is divided into two parts. The first part will guide you through creating an interactive program that calculates a dog's age in human years. In the second part, you will develop a program to solve quadratic equations using the quadratic formula. This exercise is designed to enhance your understanding of user input, arithmetic operations, and conditional logic in C++.

Objectives:

- To implement user input and output operations in C++.
- To perform arithmetic calculations based on user input.
- To apply conditional logic to solve problems.
- To develop skills in translating mathematical formulas into code.

Required Tools:

- A computer with a C++ compiler installed.

Part 1: Dog Age Calculator in Human Years

Task Description:

You will write a C++ program that:

1. Prompts the user to enter the name and age of their dog.
2. Calculates the dog's age in human years based on the provided age.
3. Outputs the dog's name along with its age in human years.

Steps:

1. Start by setting up a basic C++ program with the necessary headers.
2. Declare variables to store the dog's name, its age in dog years, and its age in human years.
3. Use **std::cin** to get the user's input for the dog's name and age.
4. Calculate the dog's age in human years assuming that:
 - The first two years of a dog's life count as 21 human years.
 - Each subsequent year counts as four human years.
5. Display the result using **std::cout**.

Part 2: Quadratic Equation Solver

Task Description:

Develop a C++ program that:

1. Asks the user to input the coefficients of a quadratic equation (a, b, c).
2. Uses the quadratic formula to find the roots of the equation.
3. Outputs the roots, indicating whether they are real and different, real and the same, or complex.

Steps:

1. Begin by setting up your C++ environment with the initial program structure.
2. Declare necessary variables for the coefficients and the roots.
3. Prompt the user to enter the coefficients a, b, and c.
4. Implement the quadratic formula to compute the discriminant and then determine the roots based on its value:
 - If the discriminant is positive, compute two different real roots.
 - If the discriminant is zero, compute one real root.
 - If the discriminant is negative, compute two complex roots.
5. Display the roots using appropriate messages to indicate their nature.

Part 3: Creating a Leap Year Checker in C++

Objective: Develop a C++ program that determines if a given year is a leap year. This exercise will reinforce your understanding of conditionals, input/output operations, and logical operators.

Background

A leap year is a year, occurring once every four years, that has 366 days instead of the usual 365 days. This extra day is added to the calendar on February 29th. The rules for determining whether a year is a leap year are as follows:

1. A year is a leap year if it is divisible by 4.
2. However, if the year is divisible by 100 (such as 1900 or 2100), it is not a leap year unless it is also divisible by 400.

Requirements

- The program should prompt the user to input a year.
- Ensure the year is a four-digit number.
- Implement the rules stated above to determine if the year is a leap year.
- Display the result in a clear and concise manner.

Instructions

1. **Setup:** Create a new C++ file named **leap_year.cpp**.
2. **Input Handling:**

- Prompt the user to enter a year.
 - Check if the entered year is a four-digit number. If not, prompt the user again.
3. **Leap Year Calculation:**
- Implement the leap year determination logic using conditional statements:
 - Check if the year is divisible by 4.
 - If the year is divisible by 100, check if it is also divisible by 400.
 - Use appropriate logical operators to combine these checks.
4. **Output:**
- Print whether the entered year is a leap year or not.

Task:

Write the code in **leap_year.cpp** and test it with various years to ensure accuracy. Include error handling where necessary and make sure the output is user-friendly.

Extension (Optional):

Enhance your program by:

- Allowing continuous input until the user decides to exit.
- Implementing a function to encapsulate the leap year logic, improving the readability and modularity of your code.

Reflection:

After completing the program, think about the following:

- How did the structure of conditional statements affect the flow of your program?
- What challenges did you encounter during this exercise and how did you overcome them?

Part 4: Creating a Rock, Paper, Scissors Game

Objective: Develop a C++ program that simulates a Rock, Paper, Scissors game between the user and the computer. This exercise will reinforce your understanding of conditional statements, and input/output operations.

Background: Rock, Paper, Scissors is a simple hand game usually played between two people, where each player simultaneously forms one of three shapes with an outstretched hand. The possible outcomes are:

- Rock crushes Scissors
- Scissors cuts Paper
- Paper covers Rock

Requirements:

- The program should allow the user to choose Rock, Paper, or Scissors.
- The computer randomly selects Rock, Paper, or Scissors.

- Implement the rules stated above to determine the winner of each round.
- Display the result of each round in a clear and concise manner.

Instructions:

1. **Setup:** Create a new C++ file named **rock_paper_scissors.cpp**.
2. **Input Handling:**
 - Prompt the user to enter Rock, Paper, or Scissors.
 - Check if the entered choice is valid. If not, prompt the user again.
3. **Game Logic Implementation:**
 - Use random number generation to simulate the computer's choice.
 - Implement the game rules using conditional statements to compare the user's choice against the computer's choice.
4. **Output:**
 - Print the computer's choice and the outcome of the round (win, lose, or draw).

Task:

- Write the code in **rock_paper_scissors.cpp** and test it with various inputs to ensure the game logic is accurate. Include error handling for invalid inputs and ensure the output is user-friendly.

Extension (Optional):

- Enhance your program by:
 - Implementing a score counter to keep track of wins, losses, and draws.
 - Adding additional gestures like "Lizard" and "Spock" to increase complexity.

Reflection: After completing the program, consider the following:

- How did the structure of conditional statements affect the flow and outcome of your program?
- What challenges did you encounter during this development, and how did you address them?